COMP626: Applied Math for Scientists - Project 1

The Pleiades Problem

Annice Najafi

1 Introduction: the Pleiades before the differential equation

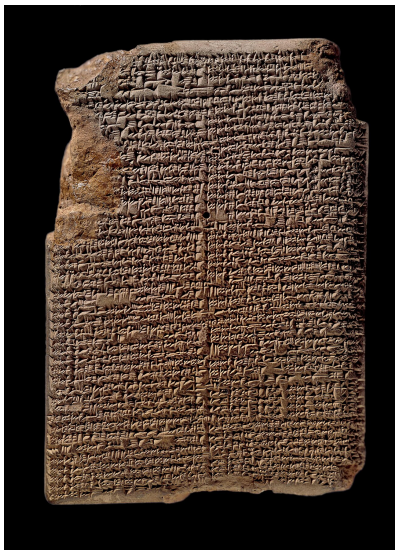
Long before the development of telescopes or modern astronomy, humans learned about the sky simply by watching it [1]. Generation after generation observed that particular stars appeared and disappeared at predictable times of the year and they learned to rely on the night sky as a natural calendar for agriculture, migration, navigation and seasonal change [2]. Among the most conspicuous objects in this ancient sky was a small, tightly packed group of stars which is now known as the Pleiades, or Messier 45 (M45). Located in the constellation Taurus approximately 445 light-years from Earth, the Pleiades is actually an open cluster containing more than a thousand stars, although only a small number are bright enough to dominate its appearance to the unaided human eye [3].



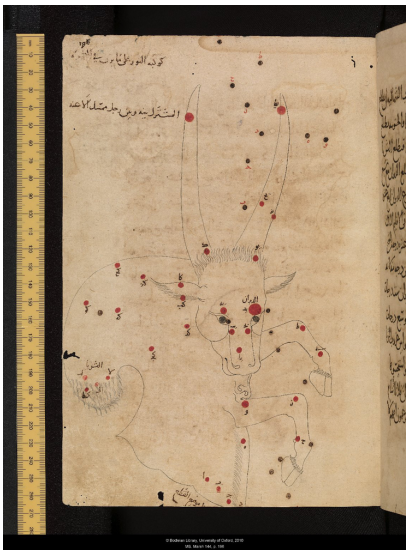
Figure 1: The Nebra sky disc, Bronze Age central Germany, usually dated to around 1600 BCE. Among the gold inlays is a tight cluster of seven dots, set apart from the scattered single stars, which is widely interpreted as the Pleiades; the disc is often described as the oldest known concrete depiction of the night sky. Photograph by Anagoria, public domain, via Wikimedia Commons. `https://en.wikipedia.org/wiki/Nebra_sky_disc`

Humans had been observing the Pleiades for thousands of years before written astronomy began. Some of the earliest surviving evidence comes from ancient Mesopotamia, where the cluster was known as MUL.MUL, essentially the Stars. Babylonian astronomical records gave the Pleiades an important position in the organization of the sky and calendar, and surviving tablets identify the

Pleiades with the Babylonian month Ajaru. The cluster also appears in Mesopotamian astronomical imagery and sources dating to antiquity [4, 5].



(a) MUL.APIN



(b) Taurus in the *Book of Fixed Stars*

Figure 2: Two written traditions that recorded the star cluster. (a) MUL.APIN, the Babylonian compendium of astronomical knowledge, whose catalogue of the stars in the path of Enlil opens with MUL.MUL, the Pleiades. Image © The Trustees of the British Museum, CC BY-SA 4.0, via Wikimedia Commons. `https://en.wikipedia.org/wiki/MUL.APIN#/media/File:MulApin-BritishMuseum.jpg` (b) Taurus as drawn in a copy of al-Ṣūfī's *Book of Fixed Stars* (composed 964 CE; this copy 1009–1010), with the tight group that Arabic astronomy called al-Thurayyā marked among the stars of the bull. Bodleian Libraries, University of Oxford, MS. Marsh 144, p. 198, CC BY 4.0. `https://commons.wikimedia.org/wiki/File:Book_of_the_Fixed_Stars_Auv0204_taurus.jpg`

The cluster also appears very early in Iranian tradition. The Pleiades are among the few stellar groups named in ancient Iranian sources, eventually becoming known in Persian as Parvin. Encyclopaedia Iranica notes that the Pleiades appear in the Younger Avesta (religious book of zoroastrians) which shows the importance of this particular group of stars in Iranian tradition at an early period [6].

Similarly, in ancient Greece, the Pleiades became particularly important practically and mythologically. The poet Hesiod, writing around the eighth or seventh century BCE, used their annual movement through the sky as an agricultural calendar. In Works and Days, he instructed farmers to begin harvesting when the Pleiades rose and to begin plowing when they set. Their disappearance and reappearance therefore helped with structuring the agricultural year [7].

At the same time, Pleiades is denoted as the Seven Sisters in greek mythology, a reference to the the seven daughters of Atlas and Pleione: Maia, Electra, Taygete, Alcyone, Celaeno, Sterope or Asterope, and Merope. The association became so deeply embedded in cultural tradition that the cluster remains known as the Seven Sisters to this day. [8]. Yet this tradition contains a very interesting underlying problem. Although the mythology describes seven sisters, most modern observers looking at the cluster with the naked eye readily distinguish only about six prominent points of light. Ancient storytellers themselves appear to have been aware of a discrepancy between

the traditional seven and the number that they were seeing. Different Greek (and other ethnicities) traditions therefore developed explanations for a faint or missing sister. For example, Merope was sometimes said to have hidden herself or become dim because she alone among the sisters married a mortal.

Interestingly, traditions involving a group of seven women or sisters associated with the Pleiades occur far beyond the ancient Mediterranean world. Among a number of Aboriginal Australian cultures, the Pleiades are associated with an extensive Seven Sisters tradition. Different communities preserve different versions. There is, however, a recurring theme that involves a group of ancestral sisters traveling across the land while being pursued by a male figure. In several traditions, the sisters eventually escape into the sky and become the Pleiades. These stories are preserved in songlines extending across enormous areas of central and western Australia. The National Museum of Australia describes the Seven Sisters story as occurring across Aboriginal Australia and documents versions maintained by communities separated by very large geographic distances [9, 10].

What makes the Seven Sisters tradition especially fascinating, however, is that the sky itself has not remained perfectly unchanged during human history. Stars are not fixed points attached permanently to a celestial sphere. They are moving through three-dimensional space, and over sufficiently long periods their apparent positions relative to one another change.

This raises an extraordinary possibility concerning the discrepancy between the traditional seven stars and the roughly six prominent stars that many people see today.

Two of the important stars in the Pleiades are Atlas and Pleione. Today Pleione lies so close to the much brighter Atlas from our perspective on Earth that their light can blend together to the unaided eye. What may appear to us as a single bright point can therefore contain the light of two distinct stars. Astronomers Ray Norris and Barnaby Norris examined the measured motions of the Pleiades and mathematically reconstructed what the cluster may have looked like in the distant past. Their analysis suggests that roughly 100,000 years ago, Pleione would have appeared farther from Atlas in the sky than it does today, potentially making the two much easier for ordinary human vision to distinguish. In their reconstruction, ancient observers may therefore have been considerably more likely to perceive seven prominent stars where many observers today perceive six [11].

If this reconstruction reflects what prehistoric observers actually saw, it produces a very interesting scenario. Imagine groups of early humans repeatedly looking toward this small and distinctive cluster tens of thousands of years ago. Seven points of light are visible, and stories, names and meanings gradually become associated with them. Those stories are passed orally from parents to children and from one generation to another. Human populations migrate and separate, eventually occupying widely distant regions of the world. Meanwhile, the stars themselves continue their almost imperceptibly slow motion.

Over thousands of generations, Pleione gradually appears closer to Atlas. Eventually, the two become sufficiently close from Earth's perspective that most observers can no longer easily distinguish them as separate naked eye stars. The sky now seems to contain six prominent points but the inherited story still says seven. Stories could then provide explanations for what the eye could no longer clearly see. A sister might have hidden herself, become faint, been separated from the others, been captured, or disappeared.

Norris and Norris therefore proposed an extraordinary hypothesis. Perhaps elements of the Seven Sisters story originated before some of the major migrations of modern humans out of Africa and were subsequently carried by human populations as they dispersed across the world. If so, aspects of the story could conceivably be tens of thousands of years old, perhaps approaching 100,000

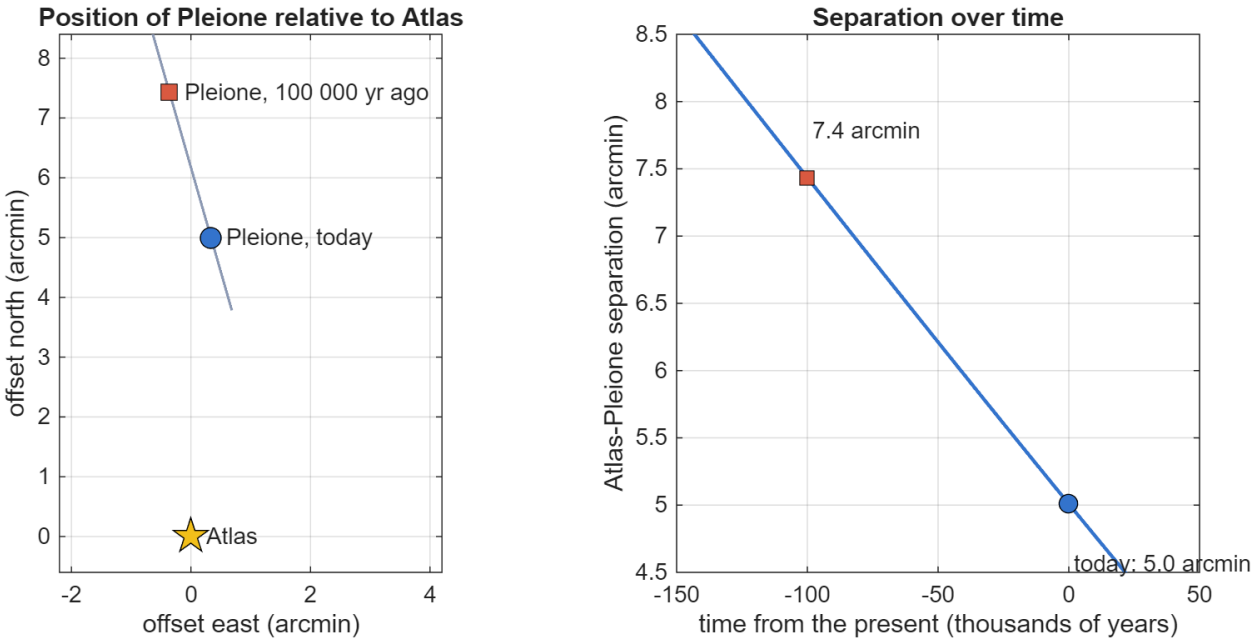


Figure 3: The mechanism, computed from catalogue data. Taking the J2000 positions and proper motions of Atlas (27 Tau) and Pleione (28 Tau) from the SIMBAD database and extrapolating the relative motion linearly, Pleione stood about 7.4′ from Atlas some 100 000 years ago against about 5.0′ today, a separation roughly half as large again. The direction and rough size of the effect are those described by Norris and Norris [11]. The extrapolation is deliberately simple: it assumes the present relative velocity has been constant, ignoring the internal dynamics of the cluster, and separability by eye depends on the brightness contrast between the two stars (V = 3.6 against V = 5.1) as well as on the angle.

years. Under this interpretation, the combination of stellar motion and oral tradition might explain two mysteries simultaneously: why widely separated cultures preserve surprisingly similar stories surrounding the Pleiades, and why traditions repeatedly emphasize seven figures even though many modern observers see only six [11].

The Seven Sisters therefore represent one of humanity's oldest continuous relationships with the natural world and tell a story transmitted across generations.

The Pleiades problem of the Bari test set follows seven bodies of masses $m_i = i$ moving in a plane under mutual gravitation — seven points of light whose motion, here, is computed rather than remembered.

2 Problem statement

1. (100 points) Consider the Pleiades problem of the University of Bari ODE Benchmark ODE Benchmark provided as equations (II.6.3) in page II-6-1 of the document, or equivalently,

(a) Write in a MATLAB Pleiades.m file the code for the right-hand-side of the E5 model.

(b) Solve Pleiades using relative tolerances (rtol), absolute tolerances (atol), and initial step size h_0 given respectively by

$$\begin{aligned} &\text{rtol} = 10^{-4}, \quad \text{atol} = \text{rtol}\,, \quad h_0 = 10^{-6} \\ &\text{rtol} = 10^{-7}, \quad \text{atol} = \text{rtol}\,, \quad h_0 = 10^{-9} \\ &\text{rtol} = 10^{-10}, \quad \text{atol} = \text{rtol}\,, \quad h_0 = 10^{-12} \end{aligned}$$

using MATLAB ODE solvers ode45, ode23, ode113, ode15s, ode23s, ode23t, ode23tb, where the first three are non-stiff ODE solvers and the last four are stiff ODE solvers.

(c) Make a plot, as in Figure II.6.2, of the evolution of x_1, y_1, using one of the solvers you used with one of the triple $(\text{rtol}, \text{atol}, h_0)$ given in the previous item.

(d) For each run report in a table (like Table II.6.2) with columns solver name, rtol (relative tolerance), atol (absolute tolerance), h_0 (initial time step), mescd (the minimum number of mixed significant correct digits in the numerical solution at the end of the integration interval)

$$\text{mescd} = -\log_{10}\left(\|y_{\text{approximated}} - y_{\text{true}}\|_\infty \Big/ \left(\frac{\text{atol}}{\text{rtol}} + \|y_{\text{true}}\|_\infty\right)\right),$$

where

$$\|(w_1, \cdots, w_n)^T\|_\infty = \max_{1 \le i \le n} \{|w_i|\},$$

scd (the minimum number of significant correct digits in the numerical solution at the end of the integration interval)

$$\text{scd} = \begin{cases} -\log_{10}\left(\|(y_{\text{approximated}} - y_{\text{true}})./(y_{\text{true}})\|_\infty\right), & \text{if } |y_{true,i}| \ge 10^{-12},\ i = 1, \cdots, n \\ -\log_{10}\left(\|y_{\text{approximated}} - y_{\text{true}}\|_\infty\right), & \text{if } |y_{\text{true},i}| < 10^{-12}, \text{for some } i \in \{1, \cdots, n\}, \end{cases}$$

steps (number of steps), accept (number of accepted steps), #f (number of function evaluations), #Jac (number of Jacobian evaluations), #LU (number of LU systems solved) and CPU (CPU time, measured as the minimum time of running the ODE solver five times).

(e) Comment on your results utilizing the table generated. Do not forget to provide the table.

(f) As deliverables, you will need to provide (via email) your source code for all the simulations, the generation of the graphic, the generation of the table, with instructions about how to run your code in interactive mode, and a text file Pleiades.txt with your comments about the table columns.

Pleiades derivation: For any body i, the net gravitational force exerted on it by all other bodies $j \neq i$ is the vector sum of individual attractive forces:

$$F_i = \sum_{j \neq i} \frac{G m_i m_j}{|r_j - r_i|^3} (r_j - r_i)$$

By substituting $G = 1$ and applying Newton's second law ($F_i = m_i r_i''$), the mass m_i cancels out, leaving the acceleration vector

$$r_i'' = \sum_{j \neq i} \frac{m_j}{|r_j - r_i|^3} (r_j - r_i)$$

3 The problem

Seven bodies of masses $m_i = i$, $i = 1, \ldots, 7$, move in a plane under mutual gravitation with the gravitational constant set to one. Newton's second law and the law of gravity gives, after the mass m_i cancels,

$$\ddot{x}_i = \sum_{j \neq i} \frac{m_j (x_j - x_i)}{r_{ij}^{3/2}}, \qquad \ddot{y}_i = \sum_{j \neq i} \frac{m_j (y_j - y_i)}{r_{ij}^{3/2}}, \qquad r_{ij} = (x_i - x_j)^2 + (y_i - y_j)^2, \tag{1}$$

which is (II.6.2). Note that r_{ij} is the squared distance meaning the exponent is 3/2 instead of 3. We can then write $z = (x^T, y^T)^T \in \mathbb{R}^{14}$ and $w = z'$ turns this into the first order system (II.6.3)

$$z' = w, \qquad w' = f(z), \tag{2}$$

of dimension 28, integrated over $0 \leq t \leq 3$ from

$$x(0) = (3,\ 3,\ -1,\ -3,\ 2,\ -2,\ 2)^T, \qquad y(0) = (3,\ -3,\ 2,\ 0,\ 0,\ -4,\ 4)^T,$$
$$x'(0) = (0,\ 0,\ 0,\ 0,\ 0,\ 1.75,\ -1.5)^T, \qquad y'(0) = (0,\ 0,\ 0,\ -1.25,\ 1,\ 0,\ 0)^T.$$

The problem is non-stiff, and during the motion several quasi-collisions occur (Table II.6.1): the closest is between bodies 1 and 7 at $t \approx 1.68$, where the squared distance falls to 0.0011. Since the right-hand side (1) behaves like $r_{ij}^{-3/2}$, these near misses are where the solution changes character abruptly, where the step size controller is forced down, and where local errors are amplified. They govern everything in the table of Section 7.

4 Part a: the right-hand side

We can take the 28 differential equations corresponding to the seven stars and their x and y positions as well as their respective horizontal and vertical velocities and put them all in one vector:

$$u = \left[x_1..x_7,\ y_1..y_7,\ x_1'..x_7',\ y_1'..y_7' \right]^T \in \mathbb{R}^{28},$$

therefore $u_1 = x_1$ and $u_8 = y_1$. In the `Pleiades.m` script, we implement Equation 2 using matrix operations. We first calculate the pairwise differences between the stars' coordinates and store them in matrices. We then set the diagonal entries of the distance matrix to infinity to exclude self-interactions ($j = i$). Finally, we calculate the accelerations in both directions using matrix-vector multiplication with the mass vector:

```
m = (1:7)';                     %masses mi = i
x  = u(1:7);
y  = u(8:14);

DX = x.' - x;                   %DX(i,j) = xj - xi
DY = y.' - y;

R32 = (DX.^2 + DY.^2).^(3/2);   %rij is the SQUARED distance
R32(1:8:end) = Inf;             %drop the i = j terms

ax = (DX ./ R32) * m;
ay = (DY ./ R32) * m;

du = [u(15:28); ax; ay];
```

If we integrate using `ode113` at tolerance rtol = atol = 10^{-13} we would be able to reproduce the reference solution of Table II.6.2 at $t = 3$ to a maximum absolute error of 2.9×10^{-11} in the 14 positions and 6.0×10^{-11} in the 14 velocities, which confirms both the formula and the ordering of the state vector.

5 Part b: numerical performance and tolerance sensitivity analysis

Using the script `run_pleiades.m` we ran seven solvers over the three triples, 21 configurations in all. We set tolerances with `odeset('RelTol',..., 'AbsTol',...)` and the initial step with the `'InitialStep'` script. We then ran each configuration three separate ways for three different purposes:

1. once with `'Stats','on'` in order to capture the printed counters with `evalc` and parse them (`pleiades_parsestats.m`) to obtain the accepted and failed steps, the Jacobian evaluations, the LU decompositions and the linear solves, as well as the end-point solution used for the accuracy measures;
2. once through a counting wrapper around the right-hand side, which gave us an exact count of function evaluations. MATLAB prints large counters in `%g` format and therefore loses digits above 10^5;

3. five more times with statistics off and the plain right-hand side, taking the minimum wall-clock time as the CPU column, as part d requires.

One issue encountered during the timing tests was that calling a MATLAB ODE solver without output arguments automatically invokes `odeplot`, which plots the solution during integration and significantly increases execution time. An earlier version of the timing loop unintentionally included this overhead and reported 60.8 s for `ode23s` at rtol $= 10^{-10}$ instead of the actual 11.0 s. This resulted in execution times being overestimated by approximately a factor of six. To avoid this issue, all solver calls in the final code explicitly request output arguments.

The end-point solution of every run is stored in `pleiades_results.mat` as a 21×28 matrix. Therefore, the accuracy measures can be recomputed under a different convention without repeating any integration.

6 Part c: evolution of x_1 and y_1

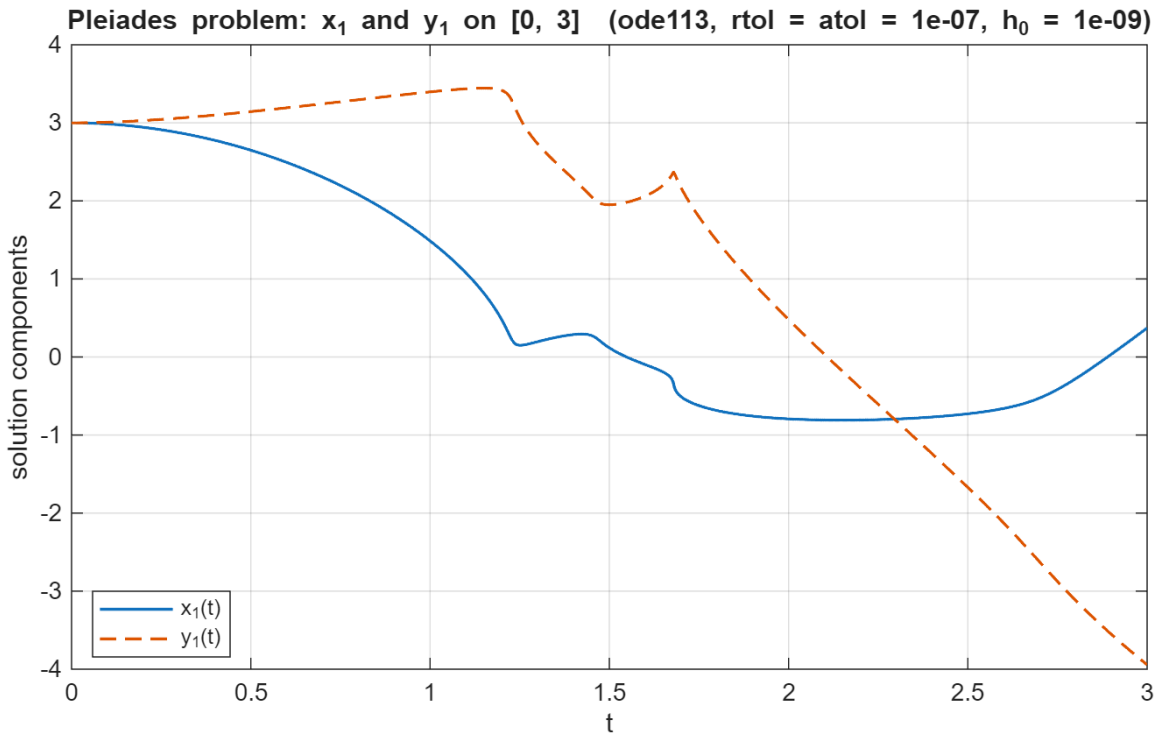


Figure 4: Solution components x_1 and y_1 over $[0, 3]$, computed with `ode113` at rtol $=$ atol $= 10^{-7}$, $h_0 = 10^{-9}$ (`pleiades_figure.m`). The kinks at $t \approx 1.23$, 1.46 and 1.68 are the three quasi-collisions of Table II.6.1 that involve body 1. The end values $x_1(3) = 0.3706$ and $y_1(3) = -3.943$ agree with Table II.6.2.

7 Part d: the table of run characteristics

The two accuracy measures

Both are evaluated at $t = 3$ against the reference solution of Table II.6.2, using the definitions given in the assignment:

$$\text{mescd} = -\log_{10}\left(\frac{\|u - u_{\text{ref}}\|_\infty}{\frac{\text{atol}}{\text{rtol}} + \|u_{\text{ref}}\|_\infty}\right), \qquad \text{scd} = -\log_{10}\left\|\frac{u - u_{\text{ref}}}{u_{\text{ref}}}\right\|_\infty, \tag{3}$$

where $\|\cdot\|_\infty$ is the largest absolute value over the components and the division in scd is element by element. (The second form of scd, $-\log_{10}\|u - u_{\text{ref}}\|_\infty$, applies when some $|u_{\text{ref},i}| < 10^{-12}$. Since all reference components are of order one, the relative form is used.)

Two details are important when interpreting the accuracy measures.

First, mescd is not calculated separately for each component. Instead, both norms are taken over the entire vector and the largest absolute error is scaled by a single value:

$$\frac{\text{atol}}{\text{rtol}} + \|u_{\text{ref}}\|_\infty = 1 + 5.2251 = 6.2251.$$

For these runs, mescd therefore measures the largest absolute error in terms of decimal digits, with an additional constant of $\log_{10}(6.2251) = 0.794$. This definition is useful when a solution component approaches zero, since relative errors can become very large even when the absolute error remains small.

Second, mescd and scd use different sets of components, following the conventions of the reference test set. For mescd, all 28 components are included, as the test set specifies that all the components of the solution are taken into account (page II-ii). For scd, only the first 14 components, corresponding to the positions, are used, as specified for the Pleiades problem (page II-6-3). These component sets are defined by `P.imescd` and `P.iscd` in `pleiades_setup.m`.

Using only the 14 position components for mescd would increase the reported values by approximately 0.1 to 0.6 digits. For example, the mescd value for `ode45` at rtol $= 10^{-4}$ would increase from 2.11 to 2.37. This happens because the largest absolute errors generally occur in the velocity components.

It is also important to note that the original test set uses a different definition of mescd (formula II.2, page II-ii), which calculates the error component by component:

$$-\log_{10}\|e \,./\, (\text{atol} + \text{rtol} \cdot u_{\text{ref}})\|_\infty .$$

As a result, the mescd values reported in Table II.6.3 cannot be directly compared with those obtained in this study. However, since the definitions of scd are the same in both cases, those results can be compared, as discussed in Section 8.

We note that in part d, the column labeled "#LU (number of LU systems solved)" may be somewhat ambiguous, as it could refer to either LU factorizations or the linear systems solved using those factorizations. MATLAB reports these quantities separately, while the reference test set reports the number of LU decompositions (page II-ii). For clarity, we included both quantities in our results. This distinction is important for understanding the efficiency of implicit solvers. For a dense system of size n, an LU factorization requires $O(n^3)$ operations, whereas solving a system using an existing

factorization requires only $O(n^2)$ operations. By reusing factorizations for multiple solves, implicit methods can significantly reduce their computational cost.

The table

solver	rtol	atol	h_0	mescd	scd	steps	accept	#f	#Jac	#LU	#solve	CPU
`ode45`	10^{-4}	10^{-4}	10^{-6}	2.11	1.14	127	106	763	–	–	–	0.0057
`ode45`	10^{-7}	10^{-7}	10^{-9}	5.70	4.77	371	371	2 227	–	–	–	0.0139
`ode45`	10^{-10}	10^{-10}	10^{-12}	9.51	8.84	1 465	1 465	8 791	–	–	–	0.0523
`ode23`	10^{-4}	10^{-4}	10^{-6}	2.47	1.82	464	460	1 393	–	–	–	0.0089
`ode23`	10^{-7}	10^{-7}	10^{-9}	5.54	4.84	4 584	4 584	13 753	–	–	–	0.0808
`ode23`	10^{-10}	10^{-10}	10^{-12}	8.54	7.84	45 793	45 793	137 380	–	–	–	0.8093
`ode113`	10^{-4}	10^{-4}	10^{-6}	2.07	1.48	363	343	707	–	–	–	0.0056
`ode113`	10^{-7}	10^{-7}	10^{-9}	5.31	4.33	679	657	1 337	–	–	–	0.0104
`ode113`	10^{-10}	10^{-10}	10^{-12}	8.36	7.71	1 128	1 110	2 239	–	–	–	0.0172
`ode15s`	10^{-4}	10^{-4}	10^{-6}	1.40	0.74	522	463	795	1	112	766	0.0087
`ode15s`	10^{-7}	10^{-7}	10^{-9}	4.27	3.59	1 397	1 327	1 612	1	178	1 581	0.0219
`ode15s`	10^{-10}	10^{-10}	10^{-12}	6.70	6.04	4 125	4 048	4 610	1	351	4 579	0.0509
`ode23s`	10^{-4}	10^{-4}	10^{-6}	1.98	1.01	592	590	18 295	590	592	1 776	0.1112
`ode23s`	10^{-7}	10^{-7}	10^{-9}	4.14	3.21	5 875	5 875	182 138	5 875	5 875	17 625	1.1018
`ode23s`	10^{-10}	10^{-10}	10^{-12}	6.16	5.23	58 702	58 702	1 819 765	58 702	58 702	176 106	11.2093
`ode23t`	10^{-4}	10^{-4}	10^{-6}	1.49	0.61	808	807	1 340	1	136	1 311	0.0113
`ode23t`	10^{-7}	10^{-7}	10^{-9}	3.51	2.67	8 160	8 160	8 360	1	154	8 329	0.0668
`ode23t`	10^{-10}	10^{-10}	10^{-12}	5.48	4.73	81 309	81 309	81 498	1	159	81 467	0.6390
`ode23tb`	10^{-4}	10^{-4}	10^{-6}	1.59	0.74	643	642	2 101	1	133	2 715	0.0171
`ode23tb`	10^{-7}	10^{-7}	10^{-9}	3.60	2.82	6 419	6 419	19 268	1	155	25 656	0.1422
`ode23tb`	10^{-10}	10^{-10}	10^{-12}	5.62	4.68	64 257	64 257	192 614	1	164	256 840	1.4192

steps = accepted + failed attempts. CPU in seconds, minimum of five runs, MATLAB R2025a on Windows 11. A dash marks a counter the solver does not report: `ode45`, `ode23` and `ode113` are explicit and form neither Jacobians nor linear systems.

8 Part e: discussion

The three non-stiff solvers consistently outperform the stiff solvers at all three tolerance levels. At rtol = 10^{-10}, `ode113` achieves 7.71 correct digits in just 0.017 s with 2,239 function evaluations. In comparison, `ode23s` requires 11.2 s and approximately 1.82 million function evaluations, while achieving only 5.23 correct digits. This makes `ode23s` roughly 650 times slower, despite producing about 2.5 fewer correct digits.

We note that this difference does not necessarily mean that the stiff solvers are less effective. They perform additional computations, including Jacobian evaluations, matrix factorizations, and Newton iterations, that provide little benefit for this non-stiff problem. In the Pleiades system, the step size is primarily determined by the accuracy required during close encounters between the stars, instead of numerical stability restrictions. As a result, solvers designed for non-stiff problems are better suited to this system and achieve greater accuracy with significantly less computational effort.

Reducing the tolerance by six orders of magnitude leads to a substantial increase in the number of steps required by most solvers. The step counts increase by the following factors:

`ode23` : 98.7×, `ode23t` : 100.6×,
`ode23tb` : 99.9×, `ode23s` : 99.2×,
`ode45` : 11.5×, `ode113` : 3.1×,
`ode15s` : 7.9 × .

These results are consistent with the classical relationship steps $\sim \text{tol}^{-1/(q+1)}$, where q is the order of the numerical method. For a second-order method ($q = 2$), this relationship predicts a 100-fold increase in the step count, which closely matches the results for `ode23`, `ode23t`, `ode23tb`, and `ode23s`. For `ode45`, using $q = 4$ predicts an increase of approximately 15.8 times, compared with the observed factor of 11.5.

Among all the solvers, `ode113` shows the smallest increase in step count, at only 3.1 times. Its ability to adjust the method order allows it to maintain efficiency as the tolerance becomes more restrictive. At rtol = 10^{-10}, it is also the fastest solver in the comparison. Meanwhile, `ode45` achieves the highest accuracy, with scd = 8.84, while requiring only 0.052 s of computation time.

At rtol = 10^{-4}, none of the solvers achieves two correct digits, with scd values ranging from 0.61 to 1.82. This indicates that the numerical errors affect even the leading digits of the final positions. The three close encounters involving body 1, shown in the figure in part c, can amplify errors accumulated during integration and contribute to the loss of accuracy.

This behavior highlights an important distinction between local and global error. The specified tolerance controls the estimated local error at each integration step, but does not guarantee the same level of accuracy in the final solution. Because the Pleiades system is sensitive to accumulated numerical errors, a tolerance of 10^{-4} does not necessarily produce four correct digits. Therefore, the accuracy of each solver should be evaluated by comparing its results with a reliable reference solution.

From rtol = 10^{-4} to 10^{-10}, `ode45` improves from scd = 1.14 to 8.84, about 1.3 digits per decade, and `ode23` from 1.82 to 7.84. The stiff solvers gain less: `ode23t` goes from 0.61 to 4.73, about 0.7 digits per decade, and stays two to four digits behind the explicit solvers at equal tolerance.

The mescd values are higher than the scd values in all 21 runs, with differences ranging from 0.59 to 0.98 digits. Because the two measures in Equation 3 use different components and scaling methods, this difference is not constant.

The mescd measure considers the largest absolute error across all 28 components and scales it by a single constant, 6.2251. In every run, the largest absolute error occurs in a velocity component, most commonly x_5' or x_1'. These components tend to have relatively large values and change rapidly throughout the simulation.

In contrast, scd considers only the 14 position components and divides each error by its corresponding reference value. The largest relative error occurs in x_1, with a reference value of 0.3706, in most runs, and in x_5, with a reference value of 0.3426, in the remaining runs. These are the two smallest reference positions in absolute value, making their relative errors more sensitive to a given absolute error.

As a result, the difference between mescd and scd depends on how the numerical errors are distributed between the position and velocity components. Since this distribution changes across solvers and tolerance levels, the gap between the two measures also varies. Despite these differences, both measures give the same ranking of solvers in this experiment. However, their results could differ substantially if a reference position were close to zero at $t = 3$.

The solvers `ode15s`, `ode23t`, and `ode23tb` each evaluate the Jacobian only once during the entire integration, regardless of the tolerance. They also perform relatively few LU factorizations, ranging from 112 to 351. This suggests that the Newton iterations converge efficiently for this non-stiff problem and allow MATLAB to reuse the same Jacobian without repeatedly recomputing it. For example, at rtol = 10^{-10}, `ode23t` performs 81,467 linear solves using only 159 LU factorizations, with 81,498 function evaluations, approximately one per step.

In contrast, `ode23s` requires considerably more computation. Its Rosenbrock method uses the Jacobian directly in its numerical formula, and the results show that a new Jacobian and LU factorization are computed at every step. At rtol = 10^{-10}, the solver performs 58,702 Jacobian evaluations, 58,702 LU factorizations, and 176,106 linear solves, corresponding to three solves per step, one for each stage.

For this 28-dimensional system, approximating each Jacobian numerically requires approximately 28 additional evaluations of the right-hand-side function. This accounts for much of the approximately 1.82 million function evaluations reported for `ode23s`. In addition, repeatedly performing LU factorizations, which require $O(n^3)$ operations for a dense system of size n, adds substantial computational cost. These factors explain why `ode23s` is the slowest solver in the comparison.

Providing an analytical Jacobian could reduce the number of function evaluations by avoiding the additional evaluations needed for numerical approximation. However, it would not eliminate the cost of repeated LU factorizations. Therefore, `ode23s` would likely remain less efficient than the other solvers for this non-stiff problem.

The difference between attempted and accepted steps is largest at rtol = 10^{-4}. At this tolerance, `ode45` rejects 21 out of 127 attempted steps, while `ode15s` rejects 59 out of 522. As the tolerance becomes more restrictive, the number of rejected steps decreases, reaching zero for most solvers at rtol = 10^{-10}.

We can explain this behavior through how the solvers adjust their step sizes. With a larger tolerance, the solvers initially attempt larger steps, which may produce excessive errors during close encounters between the stars. These steps are then rejected, forcing the solvers to reduce their step sizes and try again. With tighter tolerances, the solvers already take smaller steps which makes it less likely that the estimated error will exceed the allowed limit.

The initial step sizes of 10^{-6}, 10^{-9}, and 10^{-12} are much smaller than necessary at $t = 0$, where the solution is relatively smooth. However, the adaptive solvers adjust their step sizes within the first few steps such that the small initial values have little effect on the overall computation. Although they may introduce a few additional steps, they also help prevent the solver from starting with a step that is too large. Therefore, the initial step size is unlikely to be a major factor in the performance differences observed in the table.

8.1 9. Comparison with the test set (Table II.6.3):

Only the scd values can be directly compared between the two tables, for the reasons explained in Section 7. At rtol = 10^{-10}, the reference test set reports scd values of 5.72 for DASSL, 7.39 for GAMD, 6.94 for MEBDFI, 7.55 for PSIDE-1, 9.20 for RADAU, and 5.20 for VODE. In comparison, our MATLAB results are 8.84 for `ode45`, 7.84 for `ode23`, 7.71 for `ode113`, 6.04 for `ode15s`, 5.23 for `ode23s`, 4.73 for `ode23t`, and 4.68 for `ode23tb`.

The MATLAB non-stiff solvers perform well compared with those in the reference test set. Among the reported methods, `ode45` is surpassed only by RADAU, while `ode113` and `ode23` achieve higher

scd values than both MEBDFI and PSIDE-1. In contrast, the MATLAB stiff solvers generally produce lower accuracy, with several performing near or below the lower end of the reference results. This is consistent with our earlier observation that non-stiff solvers are better suited to the Pleiades problem.

However, the CPU times from the two tables should not be compared directly because the experiments were conducted on different hardware and used different timing procedures. The reference test set reports the median of ten runs, whereas part d requires the minimum time from five runs.

9 Conclusion

For the Pleiades problem, `ode113` and `ode45` are the most suitable solvers based on their accuracy and computational efficiency. Among the methods tested, `ode113` provides the best balance between accuracy and computational cost, while `ode45` achieves the highest accuracy at a relatively low computational cost. A relative and absolute tolerance of 10^{-10} is recommended when high accuracy in the final positions is required.

The stiff solvers are generally less efficient for this problem, particularly `ode23s`, which requires substantially more computational effort without providing comparable accuracy. Furthermore, the results obtained with rtol $= 10^{-4}$ should be interpreted with caution, as they generally provide fewer than two correct digits and may not be sufficiently accurate for reliable quantitative analysis.

10 Part f: deliverables and how to run them

File	Role
`Pleiades.m`	(a) right-hand side, 28 equations
`pleiades_setup.m`	problem data, reference solution, solver and tolerance lists
`pleiades_accuracy.m`	mescd and scd of (3)
`pleiades_parsestats.m`	reads the counters printed by `odeset('Stats','on')`
`run_pleiades.m`	(b) the 21 runs and their characteristics
`pleiades_figure.m`	(c) the plot of x_1 and y_1
`pleiades_table.m`	(d) formats and writes the table
`project1.m`	runs (b), (c) and (d) in one go
`pleiades_table.txt`	the generated table
`pleiades_x1y1.png/.fig`	the generated figure
`pleiades_results.mat`	all results, plus the end-point solution of every run
`Pleiades.txt`	(e,f) comments on the columns and on the results
`README.txt`	instructions, including interactive use
`Report.tex/.pdf`	this report
`make_report_table.py`	turns `pleiades_table.txt` into `report_table.tex`

Everything is reproduced by one command in the folder containing the files:

```
>> project1
```

which takes about 25 seconds, `ode23s` at rtol $= 10^{-10}$ accounting for half of it. The table of Section 7 is typeset from the generated `pleiades_table.txt` by `make_report_table.py`, so the numbers in this report are never transcribed by hand.

Interactively, the pieces are independent:

```
>> P = pleiades_setup();                % problem data
>> du = Pleiades(0, P.u0)               % evaluate the right-hand side

>> opts = odeset('RelTol',1e-7,'AbsTol',1e-7,'InitialStep',1e-9);
>> [t, u] = ode113(@Pleiades, [0 3], P.u0, opts);
>> [mescd, scd] = pleiades_accuracy(u(end,:).', P.uref, 1e-7, 1e-7)

>> R = run_pleiades();                  % full sweep, 5 timing repeats
>> R = run_pleiades('Repeats', 1);      % quick version
>> R(strcmp(R.solver,'ode15s'), :)      % R is a MATLAB table
>> sortrows(R, 'CPU')

>> pleiades_table(R)                    % print and write the table
>> pleiades_figure()                    % ode113, rtol = 1e-7
>> pleiades_figure('ode45', 3)          % any solver and triple 1, 2 or 3
```

`README.txt` documents these in full, including how to recompute the accuracy measures under a different component convention from the stored end-point solutions, without repeating any integration.

References

[1] Hugh Thurston. *Early astronomy.* Springer Science & Business Media, 1994.

[2] James S Edgar. *Observer's Handbook 2020.* Royal Astronomical Society of Canada, 2019.

[3] NASA Science. Messier 45 (The Pleiades). Hubble Messier Catalog. Describes M45 as an open star cluster in Taurus approximately 445 light-years from Earth, and notes that it has been observed since antiquity.

[4] British Museum. Astronomical tablet K.8538. The museum identifies the stellar group represented on this Mesopotamian tablet as the Pleiades.

[5] British Museum. Babylonian astronomical tablet 1885,0430.15. The tablet associates the Babylonian month Ajaru with the Pleiades.

[6] D. N. MacKenzie. Constellations. Encyclopaedia Iranica. Discusses early Iranian names for stellar groups, including the Pleiades and the later Persian name Parvīn, as well as the Babylonian MUL.MUL designation.

[7] Hesiod. Works and Days. The rising and setting of the Pleiades identify the appropriate times for harvesting and plowing. Translation preserved by the Perseus Digital Library, Tufts University.

[8] The Metropolitan Museum of Art. The Pleiades. Notes the Greek mythological tradition identifying the Pleiades as the seven daughters of Atlas and Pleione.

[9] National Museum of Australia. Seven Sisters. Documents the widespread Aboriginal Australian association of the Pleiades with Seven Sisters traditions and presents the Martu Minyipuru Jukurrpa version.

[10] National Museum of Australia. Songlines: Tracking the Seven Sisters. Documents Seven Sisters traditions from the Western and Central Deserts and their preservation through songlines, art, oral tradition and knowledge of Country.

[11] Ray P. Norris and Barnaby R. M. Norris. Why are there Seven Sisters? *Advancing Cultural Astronomy*, 2021. Examines similarities between Greek and Aboriginal Australian Pleiades traditions and proposes that the proper motion of Atlas and Pleione could help explain why traditions describe seven stars although many observers now distinguish only six.